Gemma 4 Gives Its Failing Patches Full Marks

Sugam Panthi University of Southern Mississippi

github.com/Vein05/gemma4-paper-track Supplement: `rubber-stamp-supplement.pdf` Kaggle paper track

Abstract

Coding agents often decide whether a patch is finished by asking the model that wrote it. This check assumes that the model judges its own patches as well as other models' patches, and earlier work found no self-preference in Gemma 3 on contest programs. In this work, we ask Gemma 4 31B and other models to score SWE-bench Verified patches without naming their writer, and we compare each judge with judges from other model families on the same patch. Gemma gives its top score to 52 of its 85 failing patches, while the other judges score the same patches lower. Gemma also trusts other models' failing patches that make the same edit, and smaller Gemma 4 models do not share its trust in its patches. The trust stays when Gemma thinks before scoring, tests the patch in a shell, or is fine-tuned on labeled patches. When an agent shows Gemma the repository tests that its patch breaks, Gemma repairs 9 of the 52, but it also breaks a few correct patches. Gemma's verdict on a patch should be checked against tests it did not write. We release everything we ran, including every agent and judge trace, on GitHub and Hugging Face.

1 Introduction

In matplotlib issue 24570, Gemma 4 31B wrote a patch that fixes the reported alignment bug but also swaps the left and right cases of a shared helper. When we later asked Gemma whether this patch would pass the project's hidden tests, it gave the highest score on our 0 to 100 scale and called the patch "a simple swap" that "correctly implements the expected behavior". GLM-4.5 scored the same patch 20 because it "breaks VPacker by reversing left and right", and the hidden tests for left and right alignment fail. Coding agents make this call at the end of every task, when they decide whether to submit a patch, try again or hand the task to a person, and many agents make it by asking the model that wrote the patch.

We call this failure rubber-stamping: a judge gives its own failing patch the top score. We measure it on SWE-bench Verified issues that Gemma attempted with mini-SWE-agent, and we compare Gemma with outside judges, models from families other than Gemma that score the same patches. Gemma rubber-stamps on these issues, on another team's Gemma run on SWE-bench Lite, and on contest programs it wrote without an agent. Five large models that judge their own patches in the same setting do not. This differs from Pombal et al. (2026), who found no self-preference in Gemma 3 12B and 27B on contest programs.

The trust goes with the edit more than with the writer. Gemma also trusts failing patches from other models when they make the same edit as its own, while Gemma 4 12B and 26B-A4B, reading the 31B's patches, trust its failing patches no more than other writers' failing patches. On contest programs, each lenient model trusts its own wrong program more than other wrong programs for the same problem. Gemma is not lenient in general, since it scores other writers' failing patches slightly below the outside judges. We read this as trust in the fix itself. Gemma trusts the kind of patch it would write, and when it explains its score, it often names the defective line and calls it correct.

Trust in a fix should survive checks that Gemma runs itself, and it mostly does. When Gemma thinks before scoring, writes and runs a test in a shell, or is fine-tuned on labeled patches, most rubber-stamped patches

stay approved. Evidence that Gemma did not write works better, because it can contradict that trust. An agent can run the repository's existing tests on the changed code before and after the patch, and show Gemma the tests that break.

We make the following contributions:

- **Gemma 4 31B rubber-stamps its failing patches.** On held-out SWE-bench Verified issues, it gives the top score to 52 of its 85 failing patches, while outside judges score the same patches lower. The rate is 50 to 63% on all three data sets (Section 3.1).
- **The trust goes with the edit.** Other writers' failing patches that make Gemma's edit get extra trust from Gemma, 20 points on SWE-bench Lite, while smaller Gemma 4 models give the 31B's failing patches no measurable extra trust (Section 3.2).
- **Tests that Gemma did not write let it repair some patches.** Shown the repository tests its patch breaks, Gemma repairs 9 of the 52 rubber-stamped failing patches, against 3 after a plain second look. The same step breaks 4 correct patches (Section 3.4).

2 Setup

Patches. The main data set is 193 held-out issues, a random sample of SWE-bench Verified issues (Jimenez et al., 2023) outside SWE-bench Verified Mini (MariusHobbhahn). We call the model that wrote a patch its writer. The 4-bit competition checkpoint of Gemma 4 31B made one attempt per issue in mini-SWE-agent v1.7.0 (SWE-agent) at temperature 0, and we graded its patches with SWE-bench's grading code. We kept an issue only if its reference patch passes in our environment, which kept 193 of 200. Other writers' patches are public mini-SWE-agent submissions (SWE-bench experiments) by GLM-4.5, gpt-oss-120b, Kimi-K2, Llama-4-Maverick and Qwen3-Coder, with their official labels. We removed test files from every patch and dropped patches over 30,000 characters.

Two more data sets test whether the result depends on our run. Another team ran Gemma 4 31B on the 300 SWE-bench Lite issues, in mini-SWE-agent 2.2.1 at full precision with thinking on. On LiveCodeBench, our checkpoint wrote one program per contest problem with no agent. On the 50 Mini issues, Gemma's patches from four public harnesses (n-pelleriti), including SWE-agent (Yang et al., 2024), give the same picture (Supplement, Section C).

Judges. Nine models scored every held-out patch: Gemma 4 31B, gpt-oss-20b, Qwen3.5-9B, GLM-4.7-Flash and the five writers above. Each judge saw the issue and one patch, without its writer, and scored the chance that the patch passes the hidden tests from 0 to 100. Calls went through OpenRouter at temperature 0 with reasoning off, except for gpt-oss, which always reasons and ran at low effort. Credit limits cut the judges to seven on Lite and six on LiveCodeBench, before any score was read.

Measures. A judge's score on a failing patch says little alone, because some failing patches look right to everyone. We therefore compare each judge with the outside judges on the same patch. A judge's leniency on a patch is its score minus the outside judges' mean score, so positive leniency is extra trust that the outside judges do not share. We report how much higher Gemma's leniency is on its own failing patches than on other writers' failing patches, with 95% intervals from 2,000 bootstrap resamples of issues. The rubber-stamp rate is the share of a judge's own failing patches that it scores 100.

Plans. We committed a written plan, with the rule for reading its result, before computing each test. We mark as exploratory every analysis chosen after we saw a result.

3 Results

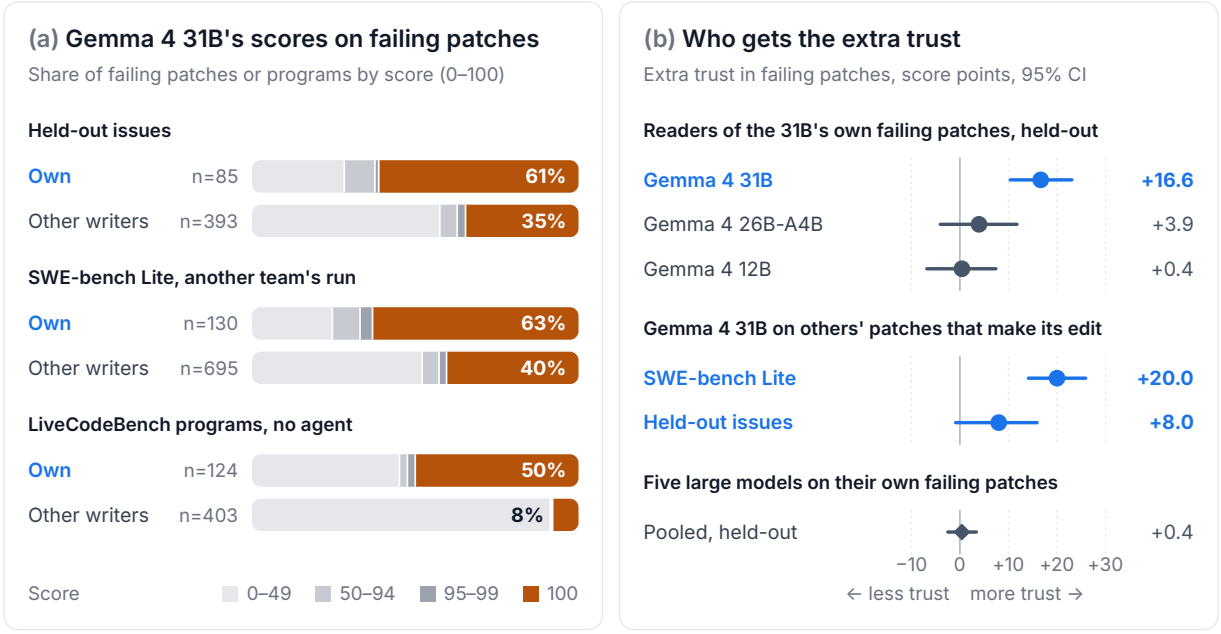


Figure 1. Gemma 4 31B approves most of its failing patches, and the extra trust goes with the edit. (a) Gemma's scores on failing patches, its own and other writers', on three data sets. (b) Extra trust in failing patches, in score points with 95% intervals: a reader's leniency on the named failing patches minus its leniency on other writers' failing patches. In the two same-edit rows, the comparison is other writers' failing patches without Gemma's edit.

3.1 Gemma rubber-stamps its failing patches

Gemma gives 100 to about half or more of its failing patches on all three data sets, and to far fewer of other writers' failing patches (Figure 1a). The outside judges do not share this trust. On the held-out issues, Gemma's leniency on its own failing patches exceeds its leniency on other writers' failing patches by 16.6 points [10.3, 23.1]. The difference is 17.8 points on Lite and 23.0 on LiveCodeBench, and both intervals lie above zero. Gemma's failing patches do look a little better to everyone, so we also compared them only with other writers' failing patches that the outside judges score the same. The held-out difference is still +13.6 [7.1, 20.3] (exploratory, Supplement, Sections C and D).

Five large models judged their own held-out patches in the same way, and none shows Gemma's leniency. Their pooled leniency is +0.4 [−2.5, 3.4] (exploratory, Supplement, Section C). Smaller open models are different. On LiveCodeBench, three of four usable small open models are lenient on their own failing programs, and so are Gemma 4 12B and 26B-A4B, while Gemma 4 31B is the most lenient of all (Supplement, Section C).

3.2 The trust goes with the edit

Gemma could trust its patches because it recognizes them as its own, or because it believes in the edit they make. Three tests point to the edit. First, we split other writers' failing patches by whether they change the same lines as Gemma's failing patch for that issue, with a Jaccard similarity of their changed lines of at least 0.5. On Lite, Gemma gives the same-edit patches 20.0 points more extra trust than the others [14.1, 25.9] (Figure 1b). On the held-out issues the difference is 8.0 points, and its interval reaches zero. We detect no extra trust in Gemma's own patch beyond these same-edit patches on either set (Supplement, Section D).

Second, Gemma 4 12B and 26B-A4B scored the 31B's held-out patches. They come from the same family and read the same style of code, but we find no measurable extra trust in the 31B's failing patches over other writers' failing patches (+0.4 and +3.9 points, both intervals include zero). Of the three sizes, only the 31B shows the extra trust. Third, on LiveCodeBench, each lenient model trusts its own wrong program more than other wrong programs for the same problem, and Gemma 4 31B does so by 39.0 points [30.7, 47.7]. For six of seven judges, being wrong on a problem does not make them more lenient toward other writers' wrong programs (Supplement, Section D).

Gemma's explanations fit this reading. Two coders, both Claude models working apart, read Gemma's answers on 31 of its rubber-stamped held-out patches. In all 18 cases where the defect sits on one or two marked lines, Gemma's answer endorses the defective line itself (Supplement, Section D).

The trust is not limited to Gemma's own fixes. On Lite, Gemma trusts Claude 4 Sonnet's failing patches as much as its own (+1.7 [−5.3, 8.7]). Our data cannot tell the fix Gemma would write from the fix most strong models would write, and we never varied the writer with the patch held fixed.

3.3 Checking harder through Gemma leaves the trust in place

If the trust is a belief about the fix, a check that Gemma runs should inherit it. We tested four such checks.

Check run by Gemma	What happens to Gemma's failing patches
Thinking before scoring (Mini issues)	still scores 62% of its failing patches 100, against 61% without thinking
Its own reproduction script, written while fixing the issue	the last run ends without an error in 63 of 73 failing runs that ran one
A shell with the patch applied; Gemma must write a test that fails without it	still scores 80% of the 51 it had scored 100 at 100
LoRA fine-tuning on 990 labeled patches from 184 other issues	still accepts 48 of the 51 it had scored 100, against 50 before

The shell check is the most direct. Gemma had a working environment and 15 steps, and in 61% of its runs on failing patches it wrote a test that fails without the patch and passes with it. Its test checks the behavior Gemma believes is right. The shell lowers Gemma's score on about one in five of these failing patches, but also on one in twenty of the passing ones, so it separates them only slightly (AUROC 0.58). The LoRA adapter did not reduce the failing patches Gemma accepts on new issues by a measurable amount (−4.2 points [−8.3, 0.0], one seed, Supplement, Section E).

3.4 Tests that Gemma did not write let it repair some patches

The repository's existing tests encode what the project's developers expected. We gave the agent a check that needs no hidden test and no second model. It picks the existing tests whose files or functions match the changed code, runs them twice before the patch and once after, and holds back the patch if a test that passed both times now fails or disappears. A held-back patch gets one more Gemma run in mini-SWE-agent, with up to 50 steps. In one condition Gemma sees the broken tests and their output, and in the other it gets a plain second look at its patch. We graded every result with the hidden tests (Supplement, Section F).

The check holds back 26 of Gemma's 85 failing patches, including 15 of the 52 it scored 100. In 14 of those 15, the broken test passes with the reference patch, so these are real regressions that Gemma missed. Over all 26 held-back failing patches, Gemma repairs 14 when it sees the broken tests and 6 after a second look. Eight were repaired only with the tests shown and none only by the second look (one-sided sign test, $p = 0.004$), which meets the reading we fixed before the run.

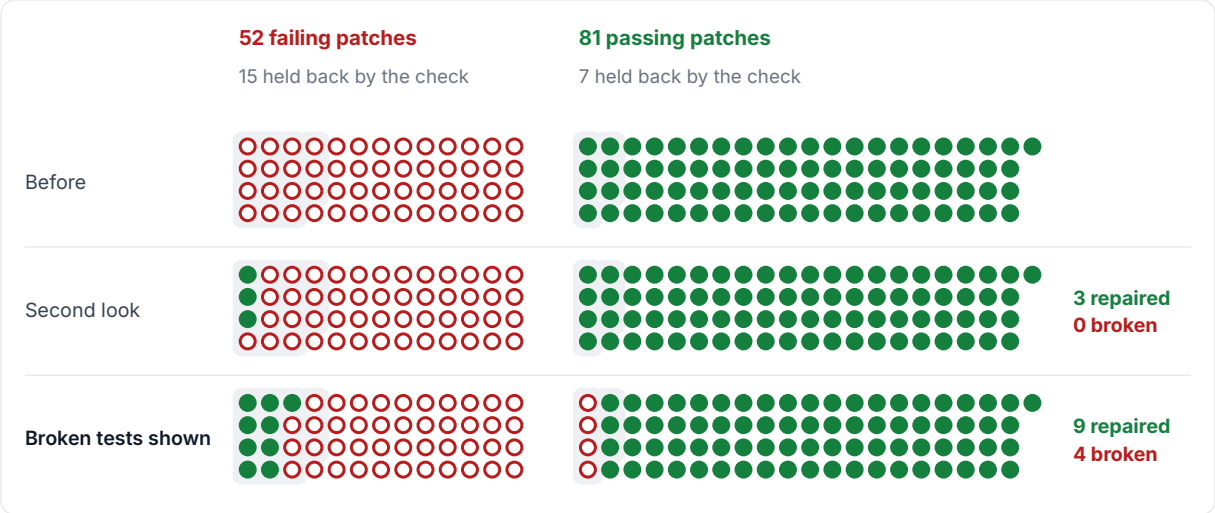


Figure 2. Showing Gemma the tests its patch breaks repairs 9 of its 52 rubber-stamped failing patches and breaks 4 of its 81 correct ones. Each dot is one of the 133 held-out patches Gemma scored 100: filled if it passes the hidden tests, hollow if it fails. The grey band marks the patches the check held back, and only these got a repair run.

The repair has a cost. Some existing tests assert the old behavior that the issue asks to change, so the check also holds back correct patches. It holds back 7 of the 81 passing patches Gemma scored 100. It even holds back 18 of the 193 reference patches. Shown such a test, Gemma often obeys it. In 3 of the 4 correct patches it broke, the broken test also fails with the reference patch, and Gemma restored the old behavior to make it pass. On sympy issue 12481 it wrote that it had "re-introduced the disjointness check to make `test_args` pass, although this contradicts the PR description". Among the 133 patches Gemma scored 100, nine repairs and four breaks leave five fewer failing patches. Over all 180 patches, 14 repairs and the same four breaks add ten passing patches.

An agent can also set Gemma's verdict aside without repairing anything. Ranking Gemma's patches by its score and then by how few model calls its run took raises the share of the top quarter that passes from 61% to 78% (Supplement, Section F), but every failing patch it submits stays wrong.

4 Related work

Judges that favor their own outputs have been studied mostly on text and short programs. Panickssery et al. (2024) link this self-preference to how well a model recognizes its own summaries. Roytburg et al. (2026) compare a judge's own wrong answers with other models' wrong answers to the same question, and much of the measured self-preference goes away. Our leniency measure also compares judges on the same patch.

Chen et al. (2025) find that stronger judges more often prefer their own wrong answer to a correct one. Pombal et al. (2026) find self-preference in rubric-based checks of programs, but not for Gemma 3 12B and 27B on LiveCodeBench. Khullar et al. (2026) find that one model rates a SWE-bench patch as more correct when it is framed as its own earlier answer. In our setting, the judge sees no authorship cue, and the patches come from full agent runs. SWE-Gym (Pan et al., 2024) and R2E-Gym (Jain et al., 2025) fine-tune verifiers that choose among attempts at one issue, while our check uses tests already in the repository.

5 Limitations

Our finding is about Gemma 4 31B on Python code. The repair result comes from one run per patch with one seed, on held-out issues we had studied before, and it changes the patch, not Gemma's score. In our held-out runs the repository's Python environment was not active, so Gemma often could not import the code while writing its patch. This lowers its resolve rate, but its rubber-stamp rate is similar on the runs without this problem and on the Lite and Mini runs, which were made elsewhere. Removing the leftover files that Gemma's patches add lowers the held-out leniency by 2.2 points [−4.6, 8.7]. Gemma as judge is the hosted model, and with the 4-bit competition checkpoint as judge we detect no difference (Supplement, Section A). We graded Gemma's patches ourselves, and in our environment 26 of 30 other-writer patches got their official label. All four misses were passes graded as failures, so we kept only issues whose reference patch passes.

6 Data, code and compute

We release everything we ran. The GitHub repository https://github.com/Vein05/gemma4-paper-track holds the paper, the supplement, all code, every analysis plan, every judge answer and the summary behind each number. The Hugging Face dataset https://huggingface.co/datasets/vein05/gemma4-paper-track-data holds the raw runs, including the trajectory of every agent run, the shell-judge, existing-test and repair runs, and every judge prompt. We leave out only text we may not redistribute: SWE-bench Lite issues and patches from another team's run, and LiveCodeBench problem statements. The project used about 74 GPU-hours, 59 on one A100 80 GB and 15 on L40S. At RunPod on-demand prices this compute costs about $122.

References

Wei-Lin Chen, Zhepei Wei, Xinyu Zhu, Shi Feng and Yu Meng. 2025. *Do LLM Evaluators Prefer Themselves for a Reason?* https://arxiv.org/abs/2504.03846.

Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen and Ion Stoica. 2025. *R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents*. https://arxiv.org/abs/2504.07164.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press and Karthik Narasimhan. 2023. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* https://arxiv.org/abs/2310.06770. ICLR 2024 (arXiv comment, https://openreview.net/forum?id=VTF8yNQM66).

Dipika Khullar, Jack Hopkins, Rowan Wang and Fabien Roger. 2026. *Self-Attribution Bias: When AI Monitors Go Easy on Themselves*. https://arxiv.org/abs/2603.04582.

MariusHobbhahn. *swe-bench-verified-mini*. https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini. Hugging Face dataset; revision b316c349; accessed 2026-10-05.

n-pelleriti. *AlphaDiana Gemma 4 agent-run datasets*. https://huggingface.co/n-pelleriti. Six Hugging Face datasets; accessed 2026-10-05.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr and Yizhe Zhang. 2024. *Training Software Engineering Agents and Verifiers with SWE-Gym*. https://arxiv.org/abs/2412.21139. ICML 2025 (arXiv comment).

Arjun Panickssery, Samuel R. Bowman and Shi Feng. 2024. *LLM Evaluators Recognize and Favor Their Own Generations*. https://arxiv.org/abs/2404.13076.

José Pombal, Ricardo Rei and André F. T. Martins. 2026. *Self-Preference Bias in Rubric-Based Evaluation of Large Language Models*. https://arxiv.org/abs/2604.06996.

Dani Roytburg, Matthew Bozoukov, Matthew Nguyen, Jou Barzdukas, Mackenzie Puig-Hall and Narmeen Oozeer. 2026. *Are LLM Evaluators Really Narcissists? Sanity Checking Self-Preference Evaluations*. https://arxiv.org/abs/2601.22548. ICML 2026 Main.

SWE-agent. *mini-SWE-agent*. https://github.com/SWE-agent/mini-swe-agent. GitHub repository; accessed 2026-10-05.

SWE-bench. *SWE-bench experiments*. https://github.com/SWE-bench/experiments. GitHub repository; patches from the swe-bench-submissions bucket; revision 40f164d5; accessed 2026-10-05.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan and Ofir Press. 2024. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. https://arxiv.org/abs/2405.15793.